
FVSpec: Real-World Property-Based Tests as Lean Challenges

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present a benchmark for evaluating AI models and agents on real-world formal
2 software verification tasks. We first scrape 11,039 property-based tests (PBTs)
3 from real-world Python repositories, then automatically translate them into Lean 4
4 specifications with `sorry` placeholders. Translating PBTs into Lean specifications
5 is challenging: it requires modeling Python semantics in Lean, inferring the logical
6 property encoded in an imperative PBT, and handling the inherent difficulties
7 of dependently-typed programming in a seldom-used language. We describe a
8 three-agent LLM pipeline for transpiling PBTs into Lean specifications, evaluate
9 coverage and quality metrics, and provide baselines for proof generation using
10 several automated and model-based approaches. Our benchmark aims to drive
11 progress on the underexplored problem of AI-assisted formal verification of real-
12 world software, which is of increasing interest as AI produces more and more of
13 the world’s code.

14 1 Introduction

15 Several AI safety proposals are premised on implementing safeguards by way of *formal verification*
16 (FV)¹ – that is, software systems that can produce and unequivocally certify mathematical proofs.
17 These systems can be split into *interactive theorem provers* (ITPs) which accept and validate user-
18 supplied proofs, and *automated theorem provers* (ATPs) which attempt (but often fail) to prove
19 theorems fully autonomously. ARIA’s Safeguarded AI [10], Guaranteed Safe AI [9], and Scalable
20 Formal Oversight [48, 36], among others, all suggest that in the future, all AI-generated software will
21 be formally verified by autonomous FV agents, who will steer ITPs in much the same way that
22 coding agents currently steer software production.

23 The only way FV can play this role is if agents can conduct verification work at least as well and as
24 quickly as they can produce code. Otherwise, FV becomes a bottleneck and will naturally wither
25 under the capital pressure [39] to iterate. There is currently little evidence that such capabilities
26 are possible. To achieve them, we would first need high-quality benchmarks to train on. Existing
27 benchmarks (see Table 1) are too small (e.g. [30]), focused on advanced mathematics rather than
28 engineering (e.g. [16]), or derived from expert-driven verification projects (as is the case in [8])
29 which are quite different from what you would get were you to verify the average software project.
30 To clarify this last point: historically, FV has been bottle-necked by the number of human expert
31 practitioners able to write proofs in an ITP², and therefore, outside of pedagogy or academic research,

¹FV forms a subset of *formal methods*, which also include PBT, model checking, and abstract interpretation.

²We were unable to find any formal estimate of the number of human formal methods practitioners today. However, as a very rough proxy, de Almeida Borges et al. [11] conducted the largest ever survey of users of any particular ITP – in this case Coq, which is now called Rocq – at 466 participants. As a point of contrast, that year’s survey by the Rust foundation of Rust software engineers had 7,156 participants [43]. Anecdotally, most ITP work today is in Rocq, Lean 4, or Isabelle.

FV was only ever applied to the most mathematically complex and safety-critical applications, such as compilers [27] or an OS kernel [26]. These projects are generally monolithic, closed systems, in contrast to the average software project which may have a frontend and backend, a database, reliance on several external black-box APIs such as an inference provider or a logging service, and an unnecessarily large dependency tree. The latter category of (normal) software is actually in many ways much more challenging to verify because it necessitates reasoning about black-box external systems and the interactions of several different components, potentially written in different languages and operating on different chips [6, 37].

Benchmark	Language	Theorems	Focus
Minif2f [53]	Lean, Isabelle, HOL Light, Metamath	488	math Olympiad
PutnamBench [46]	Lean, Isabelle, Coq/Rocq	640	math Olympiad
Fimo [29]	Lean	149	math Olympiad
DafnyBench [30]	Dafny	782	scraped Dafny proofs
ProofNet [2]	Lean	371	U math
RLM25 [38]	Lean	619	G math
Lean Workbook [51]	Lean	57,000	HS, U, G math
FormalMATH [52]	Lean	5,560	math Olympiad
LEAN-GitHub [50]	Lean	28,597	scraped Lean proofs
ProverBench [40]	Lean	325	math Olympiad
MiniCTX [20]	Lean	762	six Lean projects
LeanGeo [41]	Lean	122	HS, U, G math
IneqMath [32]	Lean	1,522	math Olympiad
FATE- $\{M, H, X\}$ [24]	Lean	350	U, G math
VeriBench [4]	Lean	857	software verification
FVAPPS [15]	Lean	17,931	software verification (from APPS)
CLEVER [44]	Lean	161	software synthesis & verification
Vericoding [7]	Lean, Dafny, Verus	12,504	software synthesis & verification
MUSTARD [21]	Lean	5,866	HS, U math
FormL4 [31]	Lean	14,510	U math
CoqStoq (Rango) [45]	Coq/Rocq	10,396	software verification
MSC-180 [28]	Lean	180	U, G math
HOLList [3]	HOL Light	29,462	U, G math
FVSpec:FV	Lean	75,005	software verification: specs from the wild

Table 1: **Prior ITP benchmarks.** HS = high-school, U = undergraduate, G = graduate. If one benchmark improves upon another, we exclude the lesser.

In light of the limitations of prior works, we generate a new benchmark, FVSpec, from public uses of property-based testing (PBT). While FV itself is rarely used in software development, PBT is a “close sibling” technology which has seen significant adoption, sometimes called “lightweight formal methods” [22]. In both FV and PBT, engineers write *specifications*: logical properties that the software must obey. For example, we might require that a Python function `isort(1st)` always returns a sorted permutation of its input `1st`. The difference lies in how this property is checked. In FV, the property is proved using mathematical techniques—this results in almost-perfect confidence, but requires proof engineering by highly expert teams [14]. In PBT, the code is randomly tested—e.g. for random values of `x`. This is a push-button process with no proof engineering, and provides only statistical assurance.

PBT is typically much cheaper than FV, and as a result has seen wider adoption. In fact, PBTs are typically seen as the first step toward FV. For example, in the interactive theorem prover ACL2s [13], any unproven theorem statement is automatically treated as a PBT until the author supplies a proof. Thus the idea of translating PBTs to theorems for FV is natural and, to an extent, industry-standard. We build on this. First, we scrape 11,039 PBTs written in Python’s Hypothesis framework [34]. Each PBT in our dataset can be seen as a formal conjecture about a piece of Python code, which the author would like to either substantiate (prove) or disprove. We refer to our scraped PBT corpus as FVSpec:PBT. To make it possible for an AI to resolve these conjectures, we translate both code and PBTs to the theorem prover Lean, which is becoming the de-facto standard in AI formal verification. We refer to our resultant corpus of Lean translations as FVSpec:FV. Once the Lean version of the conjecture is (eventually) proven, it can be called a *theorem*. Until then, it has a sorry placeholder where the proof belongs. This approach was established by Dougherty and Mehta [15] on the FVAPPS

62 verification benchmark. In FVAPPS, source programs were taken from the APPS benchmark by
 63 Hendrycks et al. [18], specs were inferred by language model and translated to Hypothesis PBTs, and
 64 then lifted to theorems. We apply the same process at scale to PBTs found in the wild.

65 We use AI agents to translate from Python to Lean, in the style of FVAPPS. This process is necessarily
 66 lossy, that is to say, it does not guarantee strict equivalence. However, strict equivalence is only
 67 needed if the goal is to aid the authors of the original software by resolving their open conjectures.
 68 Although when applicable we consider this goal a positive secondary contribution of our work, our
 69 actual motivation is to build a large benchmark of realistic and high-quality software verification
 70 exercises, representative of the diversity of code found in “normal” codebases. Thus, we don’t worry
 71 whether or not our translations are perfectly equivalent, so long as they are equivalently interesting.
 72 The result is 9,415 Lean 4 verification challenges. Each consists of four artifacts (Figure 1): first, the
 73 original Python implementation; second, the original Python PBT exercising it; third, a complete
 74 Lean implementation of the same function; and fourth, a Lean theorem statement of the property
 75 with a `sorry` for the proof. Unlike existing FV benchmarks, each problem corresponds to real-world
 76 software written by engineers with no formal verification experience. As Table 1 shows, the bulk of
 77 ITP benchmarks target mathematics rather than software.

78 Our main contributions are:

- 79 1. **FVSpec:PBT** — a corpus of 11,039 deduplicated Python property-based tests scraped from
 80 333 open-source repositories, the first large, license-clean PBT dataset drawn from ordinary
 81 software development (no competition problems, no curated ITP code).
- 82 2. **FVSpec:FV** — 9,415 Lean 4 formal verification challenge samples (mean of approx. 8
 83 theorems per sample) lifted from 2,772 of those PBTs by an agentic transpilation pipeline
 84 with iterative LSP repair, each annotated with structural faithfulness scores (median 0.65),
 85 difficulty label (62% hard) which is around 70% calibrated, and full artifact metadata.
- 86 3. **Baseline evaluations** of Claude Sonnet 4.6, Claude Opus 4.7, and GPT 5.4 showing mean
 87 70% on easies and mean 49% on hard.
- 88 4. **Open-source release** of the full pipeline (consisting of scraper, dependency extractor,
 89 formalization agent, and post-production scripts) under an MIT/Apache dual license³.

90 The rest of the paper is organized as follows. We formalize the task in Section 2. We characterize
 91 our input corpus, detailing how we scraped and deduplicated 11,039 Hypothesis tests from open-
 92 source repositories, in Section 3.1, and discuss how that corpus compares to a similar one released
 93 by the Hypothesis team [12]. Then, we describe our generation pipeline— scraping, dependency
 94 extraction, LLM-based transpilation with iterative Lean LSP repair, and post-production cleaning—in
 95 Section 3.2. We characterize the benchmark and present baseline results in Section 4, discuss threats
 96 to validity in Section 5, survey related work in Section 6, and conclude in Section 7.

97 2 Task Definition

98 Each problem in the FVSpec benchmark consists of four components: (1) the original Python
 99 implementation of the function under test, (2) the original Python PBT exercising that implementation,
 100 (3) a full Lean implementation of the same function, and (4) a Lean theorem corresponding to the
 101 PBT, with a `sorry` placeholder where the proof should go. The AI’s task is to fill in every theorem
 102 `sorry` with a Lean proof term that compiles cleanly under `lake build`; submissions are then scored
 103 by whether the proof retains any `sorry` or special-purpose axioms. Each problem also includes a
 104 URL pointing to the original source code and license information.

105 As a running example, consider `isort(1st)`. Figure 1 shows all four artifacts of a single FVSpec
 106 problem. The left column holds the original implementation (top) and the PBT exercising it (bottom);
 107 the right column holds the corresponding `Impl.lean` (top), a complete Lean port of the function,
 108 and `Spec.lean` (bottom), containing two theorems—one for sortedness, one for permutation— each
 109 with their own `sorry`. The AI’s task is to discharge both Lean `sorry` placeholders.

³release planned for after submission

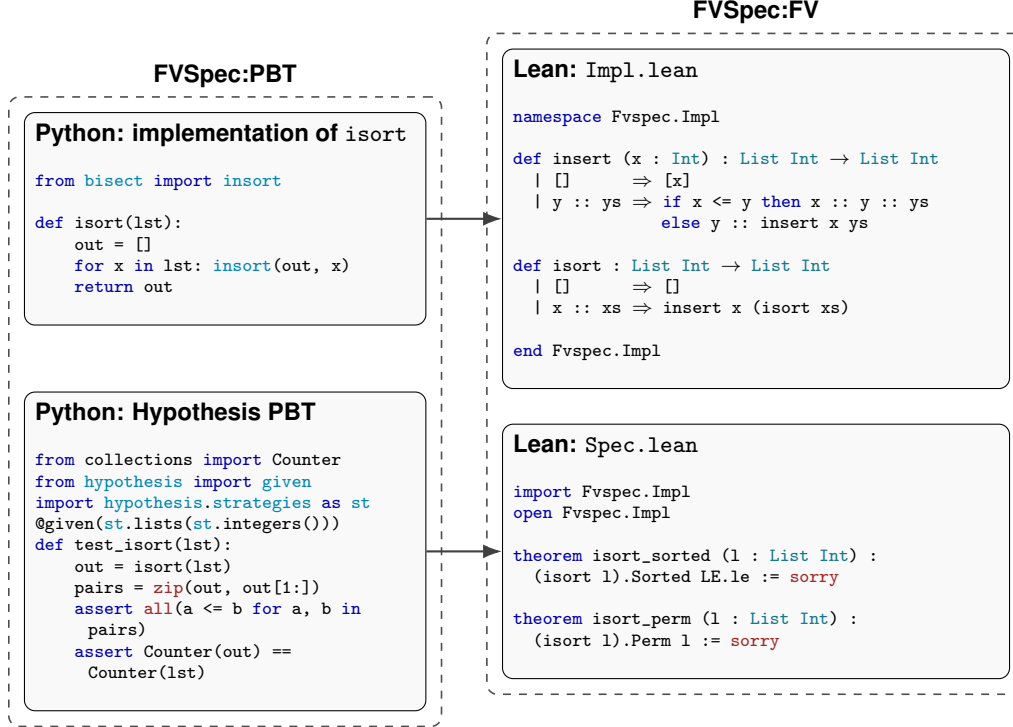


Figure 1: The four artifacts of a single FVSpec problem. Each problem is partitioned into the *PBT corpus* (FVSpec:PBT, left)—the original Python implementation and Hypothesis test we scrape from GitHub—and the *formal-verification challenge* (FVSpec:FV, right) we transpile from it: a complete Lean port of the implementation and a Lean theorem statement of the property under test. The AI must discharge both Lean sorry placeholders, one per theorem.

3 Methods

3.1 Property-Based Test Corpus (FVSpec:PBT)

We scrape real-world Python property-based tests from public GitHub repositories that depend on Hypothesis [34]. Our scraper operates in three stages. (1) *Discovery*. We crawl the Hypothesis dependency graph using PyGitHub [23] and persist 19,808 unique owner/repo pairs into a static snapshot. (2) *Licensing*. We skip repositories with an explicitly non-permissive license (e.g. AGPL, GPL, LGPL, MPL), and record the license string alongside the rest of each repository’s metadata. (3) *Extraction*. We shallow-clone each accepted repository, locate PBTs via `ripgrep`, and parse the surrounding Python to recover the transitive closure of locally-defined functions called therein.

A 24-hour run produced 54,345 raw PBTs. The GitHub dependency graph surfaces many forks of the same upstream (e.g. hundreds of forks of `pytorch` carrying an identical Hypothesis suite), so we group repositories by base name, retain the most-starred “canonical” fork per group, and additionally drop tests whose whitespace-normalized SHA-256 already appears in the canonical set. After deduplication the working corpus is 11,039 PBTs across 333 repositories owned by 281 distinct GitHub users, spanning 303 distinct upstream project names.

3.1.1 Diversity

A perennial concern with scraped benchmarks is concentration: SWE-bench [25], for instance, draws all 2,294 of its task instances from just 12 repositories, with `django` alone supplying 37% and the top three (`django`, `sympy`, `scikit-learn`) supplying 64%. Our corpus is far more spread out—it draws from 333 repositories, no single one of which contributes more than 8.7% of the PBTs and whose top ten together contribute 58.5%—and most of those repositories are unstarred personal projects rather

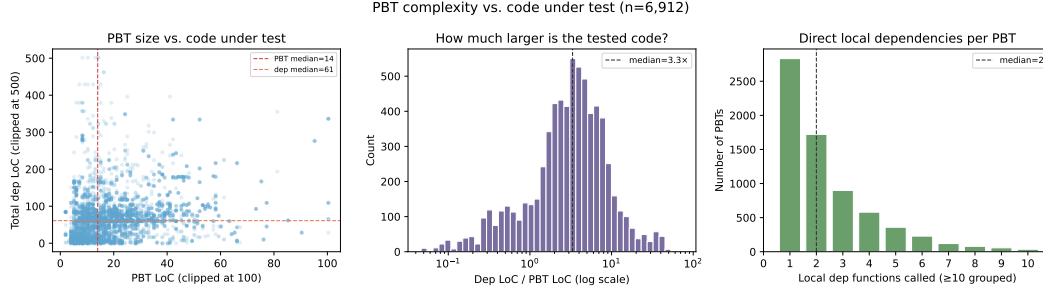


Figure 2: Three views of the relationship between PBT size and the size of the code being exercised, over the $n = 6,912$ post-deduplication PBTs whose local dependencies were recoverable. (The remaining 4,127 either exercise only standard-library or external code, or had dep extraction fail; the figure is therefore a subset of the full 11,039-PBT corpus.) (a) Per-PBT scatter of PBT LoC against the total LoC of the directly-called dependencies. (b) Histogram of the ratio between dependency LoC and PBT LoC (log scale); the median PBT exercises code $3.3\times$ its own length. (c) Histogram of the number of direct local function calls per PBT.

131 than curated showcase code (Table 2). The median PBT is just 13 LoC, but exercises around $3\times$ its
 132 own length in project code (see Figure 2).

133 To illustrate the diversity of our dataset, consider the result of drawing ten repositories from it
 134 uniformly at random:⁴ `sec-parser` (SEC-filings parser), `whenever` (datetime library), `LDAR_Sim`
 135 (methane leak-detection simulator), `bowtie` (JSON-schema test runner), `Ivy-Octernships-ML`
 136 (ML-framework scaffolding), `aaanalysis` (amino-acid analysis library), `shrinkray` (test-
 137 case shrinker), `logot` (log-output assertion library), `tokenizations` (NLP tokenisation), and
 138 `onnx-embedding` (ONNX-backed embedding library). This is broadly representative of real-world
 139 software engineering, as opposed to purely academic or competitive programming.

140 3.2 FVSpec:PBT→FV Pipeline

141 We lift each PBT in the Section 3.1 corpus (FVSpec:PBT) into a Lean (`Impl.lean`, `Spec.lean`)
 142 pair (Figure 4, bottom) in three phases: function discovery, agentic transpilation, and post-production.
 143 We call the resulting FV challenge corpus FVSpec:FV. The pipeline is modeled on FVAPS [15] but
 144 starts from PBTs rather than programming puzzles, and adds an iterative LSP-repair loop.

145 *Function discovery.* For each PBT we use `tree-sitter` to extract the function under test (FUT) and the
 146 transitive closure of its locally-defined dependencies. For our running example `isort`, this is just the
 147 four-line `def isort` body itself: `isort` comes from `bisect`, which is external and not in scope
 148 for transitive recovery. Larger PBTs in the corpus pull in tens to hundreds of lines of internal helpers
 149 (see Figure 2). When discovery fails—typically due to metaclasses, decorators, or other dynamic
 150 dispatch—we emit a stub FUT with generic type parameters so the formalization agent still receives
 151 well-formed input.

152 *Agentic transpilation.* A single formalization agent translates each sample, producing both
 153 `Impl.lean` (a computable port of the FUT and its dependencies, no `sorry`) and `Spec.lean` (one
 154 theorem per Hypothesis assertion, with proofs left as `sorry`). This is exemplified for `isort` in
 155 Figure 1. After each generation step we type-check the output via the Lean LSP (over MCP) and
 156 `lake build`; on failure, the compiler diagnostics are fed back to the agent and a revised translation
 157 is requested. The loop terminates on success or after 16 tool-iterations.

158 *Post-production.* We first extract LLM turn counts from the raw `.eval` file archives produced by the
 159 formalization agent, which runs as an Inspect AI [47] task. We then collapse runs into a single JSONL,
 160 deduplicating by sample id and tie-breaking with a quality score (structural faithfulness, theorem
 161 count, unit-tests, implementation success). Next, we re-run `lake build` on each retained sample and
 162 drop failures, which we classify into nine families: timeouts, non-wellfounded recursion, unknown
 163 identifiers, failures to synthesize, type mismatches, syntax errors, missing functions, application

⁴Reproduce with `random.seed(13); random.sample([r for r in repos if r.stars >= 10], 10)` against the 333-repo deduplicated corpus; the star floor filters out personal forks.

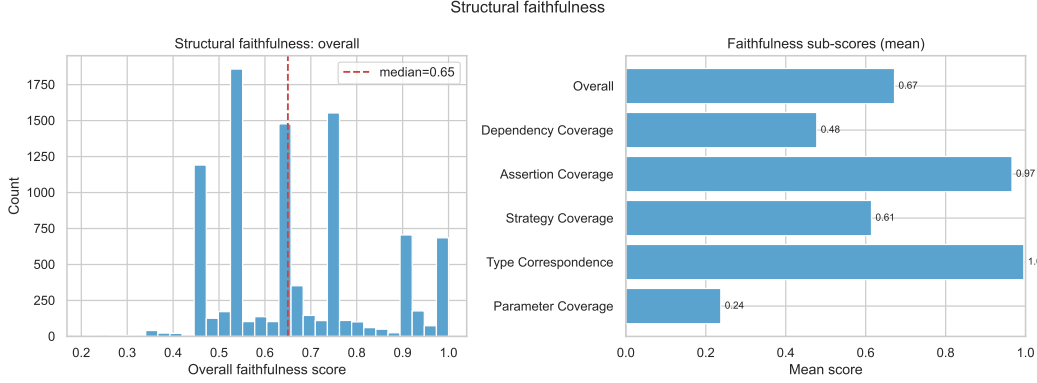


Figure 3: Structural-faithfulness scores across the published FVSpec:FV dataset. *Left*: histogram of per-sample overall scores (median 0.65; the multi-modal shape reflects the discrete denominators of the underlying ratios). *Right*: mean per-sub-metric scores. Type correspondence (1.00) and assertion coverage (0.97) are near-saturated—most translations preserve types and predicates—while parameter coverage (0.24) and dependency coverage (0.48) are the weak links, dragging the overall mean down to 0.67. The high variance in per-sample scores is exactly what the `is_canonical` flag exploits when multiple formalizations of the same PBT are available.

164 errors, and declaration errors. We then run a custom AST analyzer on the Lean artifacts to extract line
 165 counts, theorem and `def` counts, residual `sorry` and axioms. Finally, conditioning on these metrics,
 166 we assign a binary easy/hard label using `claude-haiku-4-5` as judge, together with a confidence
 167 score and a short rationale. The Haiku grader is calibrated by checking if its results are predictive of
 168 `claude-sonnet-4-6` and `gpt-5.4-2026-03-05` performance, where the grader showed approx.
 169 70% calibration

170 The structural-faithfulness score used in the merge tie-break is a fixed weighted average of five
 171 sub-scores, each measuring how much of a Python-side artifact survives translation:

$$S = 0.25 S_{\text{param}} + 0.25 S_{\text{type}} + 0.20 S_{\text{strat}} + 0.20 S_{\text{assert}} + 0.10 S_{\text{dep}} \quad (1)$$

172 where each sub-score is in $[0, 1]$: S_{param} (parameter coverage), S_{type} (type correspondence), S_{strat}
 173 (strategy coverage), S_{assert} (assertion coverage), and S_{dep} (dependency coverage). For `isort`, pa-
 174 rameter coverage rewards the Lean theorem taking a single `List Int` matching Python’s `lst`; type
 175 correspondence verifies that Python’s `list[int]` maps to `List Int`; strategy coverage tracks that
 176 `st.lists(st.integers())` ranges over arbitrary integer lists, with no missing bounds in the Lean
 177 version; assertion coverage rewards both Python assertions producing a corresponding Lean theorem;
 178 and dependency coverage rewards every named helper recovered in function discovery surviving into
 179 `Impl.lean`. Sub-scores ship alongside the overall score so consumers can re-weight or filter.

180 3.2.1 Pipeline yield

181 The published FVSpec:FV dataset contains **9,415 samples** drawn from **2,772 distinct PBTs** (Figure 4,
 182 bottom right): formalization succeeded for 9,415 of 11,039 input PBTs (25%); the rest were dropped
 183 by `lake build`. The mean sample carries 8 theorems (median 6, max 66) and median structural
 184 faithfulness 0.65 (Figure 3). A complete Lean implementation was generated for 59% of samples;
 185 the remaining 41% ship against a stub `Impl.lean`, so the theorem statement is well-typed but the
 186 body it constrains is opaque. The Claude Haiku grader splits the dataset 62%/38% hard/easy. Where
 187 a PBT yields multiple compiling formalizations (mean 3.4 formalizations per PBT, range 1–15), all
 188 are kept; the `is_canonical` field marks the highest-quality version per PBT.

189 4 Benchmark Characterization

190 The released dataset contains 9,415 Lean 4 formalization challenges (totalling 75,005 theorems)
 191 derived from 2,772 unique Python property-based tests, produced by two pipeline runs (`feb03`: 5,979
 192 samples, Claude Sonnet 4.5; `apr08`: 3,436 samples, Claude Sonnet 4.6). Each unique PBT has

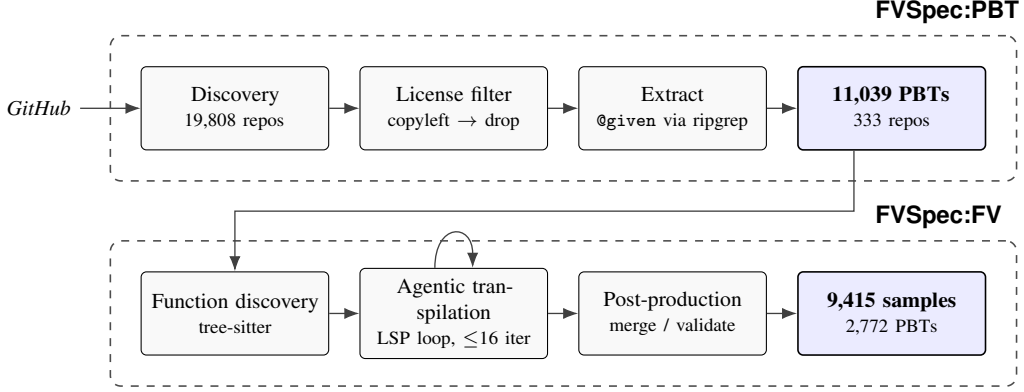


Figure 4: The full FVSpec generation pipeline (FVC = FV challenge). *Top* (FVSpec:PBT): the input PBT corpus is scraped from public GitHub repositories (Section 3.1). *Bottom* (FVSpec:FV): each PBT is lifted into a Lean (Impl, Spec) pair via tree-sitter-based function discovery, an agentic transpilation step (with the LSP-repair self-loop), and a deterministic post-production stage that validates, deduplicates, and grades the output; 11,039 PBTs enter and 9,415 FVCs (from 2,772 distinct PBTs) emerge.

193 between 1 and 15 formalizations. Among PBTs with multiple formalizations, 65% have no single
 194 dominating attempt: for each formalization in the group there exists another that scores higher on at
 195 least one sub-metric of Equation 1. In practice this means, for example, that the attempt with the best
 196 assertion coverage may have worse parameter coverage than a sibling; keeping all formalizations and
 197 flagging the best one with `is_canonical` therefore captures strictly more information than retaining
 198 only the top-scoring attempt.

199 4.1 Translation Quality

200 We evaluate translation quality using *structural faithfulness* (Equation 1), a composite metric defined
 201 in Section 3.2. The overall faithfulness distribution exhibits a clear mode above 0.5, indicating that
 202 the majority of translations preserve the essential structure of the source PBT, though a long tail of
 203 lower-quality translations remains.

204 In 59% of samples the agent generated a complete Lean implementation (`Impl.lean` contains a
 205 computable definition, not just a stub); the remaining 41% use a signature-only placeholder. Structural
 206 faithfulness is comparable across both groups (mean 0.67 vs. 0.68), confirming that specification
 207 quality is largely independent of whether the implementation was recovered.

208 The generated Lean artifacts span a wide range of complexity (Figure 10): median sample contains 6
 209 theorems with matching `sorry` counts; full distributions in D. The cost in walltime, token usage, and
 210 agent turns is shown in F

211 4.2 Difficulty Grading

212 Each sample is graded for proof difficulty by Claude Haiku 4.5, which assigns a binary label (*easy* or
 213 *hard*) along with a confidence score. Figure 8 shows the resulting split and the distribution of grader
 214 confidence. The benchmark contains a substantial proportion of both easy and hard problems, and
 215 the grader expresses high confidence for the majority of assessments. Difficulty is estimated here as a
 216 *prediction*, where we ask the grader to predict whether or not a language model agent would be able
 217 to complete the proofs successfully (considering the problem "hard" if haiku thinks not, and "easy" if
 218 haiku thinks yes).

219 Figure 9 examines the relationship between difficulty and two output characteristics: structural
 220 faithfulness and Lean output size. Hard problems tend to produce longer Lean outputs, consistent
 221 with the intuition that more complex source PBTs lead to more elaborate formalizations and more
 222 challenging proofs.

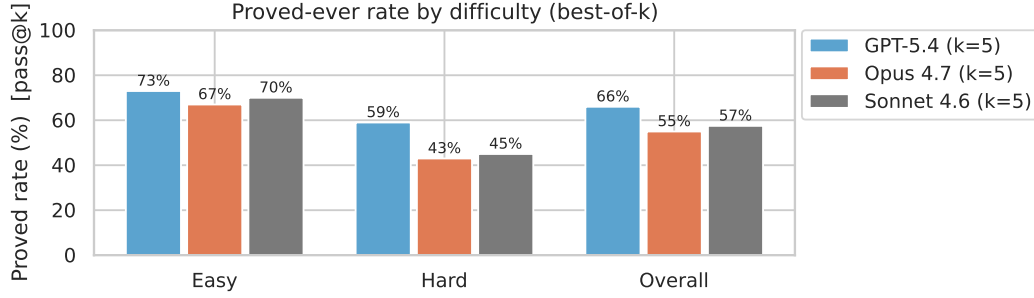


Figure 5: baseline prove rate

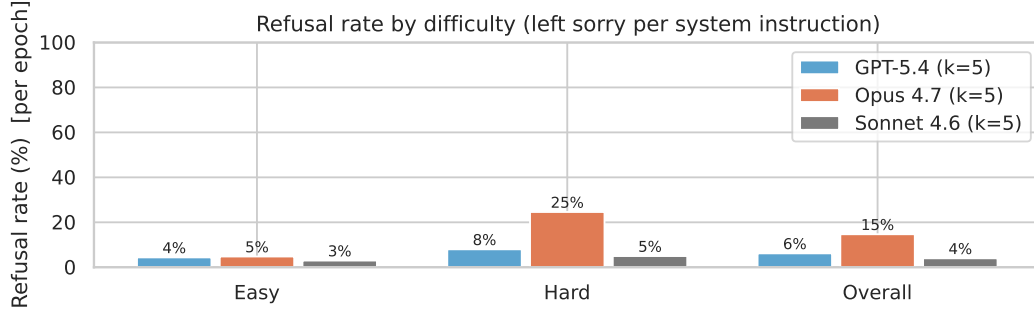


Figure 6: baseline refusal rate

4.3 Baselines

We evaluate two frontier models (Claude Sonnet 4.6 and GPT 5.4) on 100 randomly sampled easy problems and 100 hard problems (difficulty as defined in Section 4). Each model has access to the Lean LSP via MCP tools and is scored on two metrics: a binary *proved* flag (zero *sorry* remaining and *lake* build succeeds) and *partial credit* (fraction of *sorry* placeholders removed, clamped to $[0, 1]$).

5 shows that an average of 49% of hard samples were solved and an average of 70% of easy samples were solved. 6 is about when a language model believes a problem isn’t solvable.

5 Threats to Validity

The most consequential threat to FVSpec’s validity is whether theorems stated over uninterpreted axioms actually mean anything about the original Python. We address this directly in Section B.1, then catalog the remaining threats.

Translation faithfulness. Specifications are translated rather than hand-written, so some may not faithfully represent the original PBT. Our structural-faithfulness score (Section 3.2) is a heuristic: a high score does not guarantee semantic equivalence, and a low score does not necessarily indicate a useless problem.

LLM grader. Difficulty labels come from `claude-haiku-4-5`, itself a model whose judgments are imperfect and uncalibrated against human expert performance. We expose the grader’s confidence and rationale per sample so consumers can audit or override the labels.

Stub-implementation tail. A complete Lean implementation is generated for 59% of samples (Section 3.2); the remaining 41% ship against a stub, weakening the proof obligation: the theorem is well-typed but the body it constrains is opaque. These samples are tagged for easy filtering.

No human baselines. We do not measure human expert performance on FVSpec problems and therefore cannot calibrate the dataset’s difficulty distribution against the proof effort required of skilled humans.

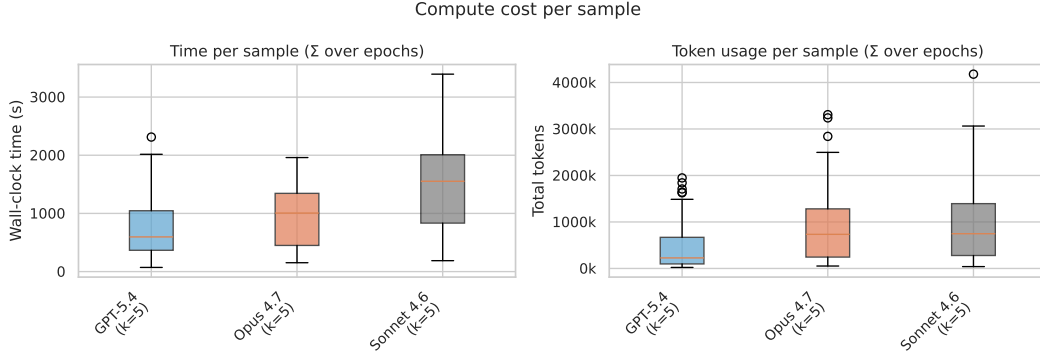


Figure 7: Compute cost per sample for each baseline run: wall-clock time (left) and token usage (right), summed across all attempts for $k > 1$ runs. Box plots show median, IQR, and $1.5 \times \text{IQR}$ whiskers.

6 Related Work

Our work extends FVAPPS [15], which translated APPS [18] coding problems into Lean theorems via inferred PBTs. We apply the same pipeline at scale to PBTs found in the wild, inferring specifications from natural language and source code.

Lean verification benchmarks. Several recent benchmarks target Lean. miniCodeProps [5] translates 177 specifications from TIP; CLEVER [44] curates 161 problems from HumanEval; Verina [49] covers code, spec, and proof generation across 189 tasks; VeriBench [42] produces 113 tasks from documented Python; the Vericoding Benchmark [7] spans Lean, Verus, and Dafny with success rates of 27–82%; and VeriEquivBench [33] uses LeetCode transformations to avoid contamination. All draw specifications from synthetic or pedagogical sources rather than properties written by practicing engineers.

Interactive theorem proving and mathematics. LeanDojo Benchmark 4 [19] (122,517 Mathlib4 theorems) and CoqStoq [45] (196,929 theorems from GitHub) are primarily mathematical and largely present in frontier model training data. FrontierMath [16] targets advanced mathematical reasoning; software verification instead demands modelling of program semantics, library APIs, and computational behavior.

FVSpec differs from prior benchmarks in three ways: (1) specifications originate from real-world PBTs written by practicing engineers; (2) the source PBTs were never formally verified, so most resulting theorems are absent from training data; and (3) we operate at thousand-scale rather than hundreds. This work is motivated by AI safety proposals premised on AI-driven formal verification, including the Guaranteed Safe AI agenda [9].

7 Conclusion

We present FVSpec, a benchmark of 9,415 Lean 4 formal verification challenges derived from 11,039 real-world Python property-based tests scraped from 333 open-source repositories. Unlike prior FV benchmarks—which draw from expert-written ITP code, competitive programming problems, or curated verification exercises—FVSpec sources its specifications directly from practicing engineers who had no formal verification intent, placing its problems genuinely out of distribution relative to models’ training data. Our agentic formalization pipeline produced 9,415 challenges from 2,772 of those PBTs on a single pass through the corpus (25% lake build success rate), with median structural faithfulness 0.65 and 62% of challenges classified as hard by a calibrated difficulty predictor. Baseline evaluations of three frontier models confirm that the benchmark is far from saturated, particularly on hard problems, leaving substantial room for progress on AI-assisted formal verification of ordinary software. Natural extensions include incorporating PBTs from other languages (Haskell/QuickCheck, C++/RapidCheck, TypeScript/fast-check) and other ITPs (Coq, Isabelle, F*, Verus), merging with the HC-2026 corpus to add runtime context, and using disproved properties as seeds for program-repair challenges—bugs the original developer’s test harness missed, now made precise in Lean.

References

- [1] M. Attanasio. Radon: Python code metrics. <https://radon.readthedocs.io/en/latest/>. Accessed 2025.
- [2] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*, 2023.
- [3] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine learning of higher order logic theorem proving. In *International conference on machine learning*, pages 454–463. PMLR, 2019.
- [4] S. Barkallah, S. Daruru, B. Miranda, L. Aniva, A. Nie, and S. Koyejo. VeriBench-FTP: A formal theorem proving benchmark in Lean 4 for code verification. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [5] D. Brandfonbrener, L. Beurerkellner, D. Romero, N. Amin, and M. Vechev. miniCodeProps: a minimal benchmark for proving code properties. *arXiv preprint arXiv:2406.11915*, 2024.
- [6] S. Buro, R. Crole, and I. Mastroeni. Equational logic and categorical semantics for multi-languages. *Electronic Notes in Theoretical Computer Science*, 352:79–103, 2020.
- [7] S. Bursuc, T. Ehrenborg, S. Lin, L. Astefanoaei, I. E. Chiosa, J. Kukovec, A. Singh, O. Butterley, A. Bizid, Q. Dougherty, et al. A benchmark for vericoding: formally verified program synthesis. *arXiv preprint arXiv:2509.22908*, 2025.
- [8] S. Chakraborty, G. Ebner, S. Bhat, S. Fakhoury, S. Fatima, S. Lahiri, and N. Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proceedings of the 47th International Conference on Software Engineering (ICSE)*, 2025.
- [9] D. Dalrymple, J. Skalse, Y. Bengio, S. Russell, M. Tegmark, S. Seshia, S. Omohundro, C. Szegedy, B. Goldhaber, N. Ammann, A. Abate, J. Halpern, C. Barrett, D. Zhao, T. Zhi-Xuan, J. Wing, and J. Tenenbaum. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. *arXiv preprint arXiv:2405.06624*, 2024.
- [10] N. A. David Dalrymple. Safeguarded ai. <https://aria.org.uk/opportunity-spaces/mathematics-for-safe-ai/safeguarded-ai/>, 2025. Accessed 25 April 2026.
- [11] A. de Almeida Borges, A. Casanueva Artís, J.-R. Falleri, E. J. Gallego Arias, É. Martin-Dorel, K. Palmiskog, A. Serebrenik, and T. Zimmermann. Lessons for interactive theorem proving researchers from a survey of coq users. *Journal of Automated Reasoning*, 69(1):8, 2025.
- [12] L. DeVoe. The Hypothesis corpus. <https://hypothesis.works/articles/hypothesis-corpus/>, 2026. Accessed April 2026.
- [13] P. C. Dillinger, P. Manolios, D. Vroon, and J. S. Moore. Acl2s: “the acl2 sedan”. *Electronic Notes in Theoretical Computer Science*, 174(2):3–18, 2007.
- [14] M. Dodds. Formally verifying industry cryptography. *IEEE Security & Privacy*, 20(3):65–70, 2022. doi: 10.1109/MSEC.2022.3153035.
- [15] Q. Dougherty and R. Mehta. Proving the coding interview: A benchmark for formally verified code generation. In *Proceedings of the LLM4Code Workshop at the 47th International Conference on Software Engineering (ICSE)*, Ottawa, Ontario, Canada, 2025. doi: 10.48550/arXiv.2502.05714.
- [16] E. Glazer, E. Erdil, T. Besiroglu, D. Chicharro, E. Chen, A. Gunning, C. Falkman Olsson, J.-S. Denain, A. Ho, E. de Oliveira Santos, O. Järvinen, M. Barnett, R. Sandler, M. Vrzala, J. Sevilla, Q. Ren, E. Pratt, L. Levine, G. Barkley, N. Stewart, B. Grechuk, T. Grechuk, S. Varma Enugandla, and M. Wildon. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024.
- [17] M. H. Halstead. *Elements of Software Science (Operating and Programming Systems Series)*. Elsevier Science Inc., 1977.

- [18] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, and J. Steinhardt. Measuring coding challenge competence with APPS. In *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- [19] R. Hsiang, W. Adkisson, R. J. George, and A. Anandkumar. LeanDojo-v2: A comprehensive library for AI-assisted theorem proving in Lean. In *The 5th Workshop on Mathematical Reasoning and AI at NeurIPS 2025*, 2025.
- [20] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024.
- [21] Y. Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. *arXiv preprint arXiv:2402.08957*, 2024.
- [22] D. Jackson and J. M. Wing. Lightweight formal methods. *IEEE Computer*, 29(4):21–22, 1996. doi: 10.1109/2.491459.
- [23] V. Jacques, L. Wan, S. Kowalik, and E. Minack. PyGitHub. <https://github.com/PyGithub/PyGithub>, 2026. Accessed 26 April 2026.
- [24] J. Jiang, W. He, Y. Wang, G. Gao, Y. Hu, J. Wang, N. Guan, P. Wu, C. Dai, L. Xiao, and B. Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- [25] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [26] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, et al. sel4: Formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, pages 207–220, 2009.
- [27] X. Leroy, S. Blazy, D. Kästner, B. Schommer, M. Pister, and C. Ferdinand. CompCert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [28] S. Li, W. Lu, X. Shi, K. Weng, H. Sun, M. Yu, T. Zhang, G. Yu, H. Liu, and L. Du. MSC-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*, 2025.
- [29] C. Liu, J. Shen, H. Xin, Z. Liu, Y. Yuan, H. Wang, W. Ju, C. Zheng, Y. Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. *arXiv preprint arXiv:2309.04295*, 2023.
- [30] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark. DafnyBench: A benchmark for formal software verification. *arXiv preprint arXiv:2406.08467*, 2024.
- [31] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, et al. Process-driven autoformalization in lean 4. *arXiv preprint arXiv:2406.01940*, 2024.
- [32] P. Lu, J. Sheng, L. Lyu, J. Jin, T. Xia, A. Gu, and J. Zou. Solving inequality proofs with large language models. In *The 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [33] X. Ma et al. VeriEquivBench: Evaluating verifiable agents on novel data without contamination. *arXiv preprint arXiv:2510.06296*, 2025.
- [34] D. R. MacIver, Z. Hatfield-Dodds, and many other contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.

- [35] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, (4): 308–320, 1976.
- [36] E. Miyazono. Formally scalable AI oversight through specifications. <https://foresight.org/resource/evan-miyazono-formally-scalable-ai-oversight-through-specifications/>, 2024. Accessed 25 April 2026.
- [37] J. T. Perconti and A. Ahmed. Verifying an open compiler using multi-language semantics. In *European Symposium on Programming Languages and Systems*, pages 128–148. Springer, 2014.
- [38] A. Poiroux, G. Weiss, V. Kunčák, and A. Bosselut. Reliable evaluation and benchmarks for statement autoformalization. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 17958–17980, 2025.
- [39] J. Rasmussen. Risk management in a dynamic society: a modelling problem. *Safety science*, 27 (2-3):183–213, 1997.
- [40] Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [41] C. Song, Z. Wang, F. Pu, H. Wang, X. Lin, J. Liu, J. Li, and Z. Liu. LeanGeo: Formalizing computational geometry problems in Lean. *arXiv preprint arXiv:2508.14644v1*, 2025.
- [42] K. Stechly et al. VeriBench: Benchmarking end-to-end formal verification. In *AI4Math Workshop at ICML*, 2025. URL <https://openreview.net/forum?id=rWkGFmnSN1>.
- [43] R. S. Team. 2025 state of rust survey results. <https://blog.rust-lang.org/2026/03/02/2025-State-Of-Rust-Survey-results/>, March 2026. Accessed 25 April 2026.
- [44] A. Thakur, J. Lee, G. Tsoukalas, M. Sistla, M. Zhao, S. Zetsche, G. Durrett, Y. Yue, and S. Chaudhuri. Clever: A curated benchmark for formally verified code generation. *arXiv preprint arXiv:2505.13938*, 2025.
- [45] K. Thompson, N. Saavedra, P. Carrott, K. Fisher, A. Sanchez-Stern, Y. Brun, J. F. Ferreira, S. Lerner, and E. First. Rango: Adaptive retrieval-augmented proving for automated software verification. *arXiv preprint arXiv:2412.14063*, 2024.
- [46] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37:11545–11569, 2024.
- [47] UK AI Security Institute. Inspect AI: Framework for large language model evaluations, 2024. URL https://github.com/UKGovernmentBEIS/inspect_ai.
- [48] M. von Hippel. <https://www.lesswrong.com/posts/SfhFh9Hfm6JYvzbby/the-scalable-formal-oversight-research-program>, 2026. Accessed 25 April 2026.
- [49] W.-D. Wu, K. Wang, R. Mangal, and I. Dillig. Verina: Benchmarking formally verified code generation. *arXiv preprint arXiv:2505.23135*, 2025.
- [50] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a versatile Lean prover. *arXiv preprint arXiv:2407.17227*, 2024.
- [51] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. *Advances in Neural Information Processing Systems*, 37:105848–105863, 2024.
- [52] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al. Formalmath: Benchmarking formal mathematical reasoning of large language models. *arXiv preprint arXiv:2505.02735*, 2025.
- [53] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.

Table 2: PBT corpus statistics.

Metric	Value
PBTs (raw scrape)	54,345
PBTs (post-dedup)	11,039
Unique repositories	333
Unique GitHub owners	281
Upstream project names	303
Top-1 / top-10 PBT share	8.7% / 58.5%
<i>Per-repo (min / med / max):</i>	
PBTs per repo	1 / 6 / 956
Stars	0 / 0 / 9,438
Forks	0 / 0 / 1,567
<i>Per-PBT (min / med / max):</i>	
PBT LoC	2 / 13 / 174
Direct fn deps	0 / 1 / 15

Table 3: Comparison with HC-2026 [12].

	FVSpec:PBT	HC-2026
Discovery	dep. graph	GitHub search
Raw PBTs	54,345	38,885
Tests (deduped)	11,039	23,139
Repos	333	1,504
Repo dedup	fork/name	is_fork
Test dedup	SHA-256	parametrization
License filter	no copyleft	none
Per-test data	source + metrics	runtime + coverage
Repo overlap (valid)	28 / 333	(8%)
Repo overlap (any)	111 / 333	(33%)

A Comparison with the Hypothesis Corpus

While we were finalizing this work, the Hypothesis maintainers released their own scraped corpus [12] (HC-2026). The two corpora target different downstream uses—HC-2026 is built to study Hypothesis *runtime* behavior (entropy consumption, predicate firing, coverage), while ours is built to feed a Lean formalization pipeline—which drives the design differences summarized in Table 3. A consequence of HC-2026’s runtime focus is that it filters out as “invalid” any repository whose PBTs may not be directly runnable without some custom setup: code that depends on external services, API keys, environment variables, or live databases. Our pipeline imposes no such requirement, since we only need a PBT’s source code to translate it into Lean. Of our 333 repositories, 28 appear in HC-2026’s valid (executable) set and 111 (33%) appear in the pre-validity superset—meaning roughly 80 of our repositories were dropped by HC-2026 for being too operationally entangled to run, consistent with our goal of producing challenging and realistic FV problems. Future work could merge the two corpora to enlarge the input pool; we did not pursue this here because the present token budget does not exhaust even our own 11,039-PBT corpus.

B Axiomatization Details

B.1 Axiomatized Interfaces for Effectful Code

The `isort` running example from Section 2 is deliberately pure. Most PBTs in our corpus are not: production code routinely touches Redis, sockets, the filesystem, the wall clock, webhook queues, and so forth. A natural objection is that any Lean theorem extracted from such a PBT cannot say anything meaningful about the live system.

The formalization agent handles this by treating the effectful world as an *uninterpreted interface*. Parts of the source essential to the property—the entities and operators it constrains—are translated into Lean structures and inductive types. Parts that are pure plumbing—the database, the queue, the system clock—are introduced as opaque axiom declarations and threaded through the theorem as parameters. The resulting theorem holds *over all interpretations* of those axioms, which is exactly the contract the original PBT was checking against the live environment.

A canonical instance is `test_workflow_job_time_to_start` from `scality/runner-manager`⁵, a GitHub Actions self-hosted runner controller. The Python PBT initializes Redis-backed model classes, runs an ORM migrator, calls `datetime.now()` to fabricate two timestamps straddling a runner-startup timeout, enqueues a webhook through a background job queue, and asserts on the resulting runner count. The FVSpec translation introduces seventeen axioms—including `Redis`, `Queue`, `State`, `State.after_enqueue`, and `get_runners`—and states the contract purely in terms of these uninterpreted operators:

```
theorem create_runner_when_above_timeout
```

⁵<https://github.com/scality/runner-manager>

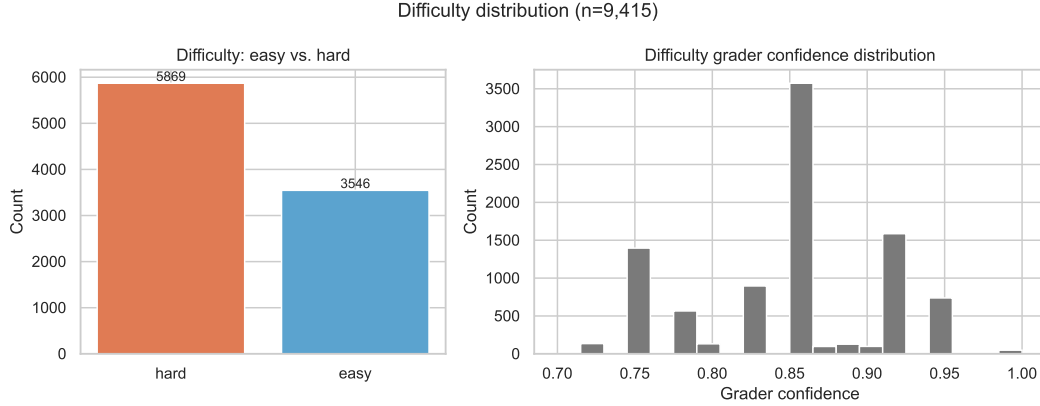


Figure 8: Difficulty distribution: easy vs. hard counts (left) and grader confidence distribution (right). n shown in title.

```

461 (initial_state : State) (state_after_enqueue : State)
462 (runner_group : RunnerGroup) (webhook_updated : WorkflowJobEvent)
463 (settings : ExtendedSettings) (created_at started_at : Time)
464 (h_initial_count : (get_runners runner_group initial_state).length = 1)
465 (h_below_max : 1 < runner_group.max)
466 (h_above_timeout : started_at - created_at > settings.timeout_runner)
467 (h_enqueue : state_after_enqueue =
468   State.after_enqueue initial_state runner_group webhook_updated settings)
469 : (get_runners runner_group state_after_enqueue).length = 2 := sorry

```

A model proving `create_runner_when_above_timeout` is reasoning about a scheduler’s state-transition semantics under a timeout precondition; the fact that the production scheduler talks to Redis is no more load-bearing in the theorem than the choice of register allocator is in a correctness proof of a sorting algorithm.

B.2 Where Axiomatization Breaks Down

The axiomatization move is not a universal solvent.

Properties about the effect itself. If the property is fundamentally about timing, memory, ordering of side effects, or wall-clock latency, an uninterpreted axiom cannot witness it. `assert response_time < 100ms` cannot become a meaningful Lean theorem because the relevant semantics is precisely what the axiomatization discards.

Effectful code disguised as pure code. The agent identifies effectful boundaries via type signatures and call patterns. When a function has a pure signature but performs side effects internally (e.g., a logger called via a global, an `lru_cache`-style hidden state), the agent will inline it as if pure. The resulting theorem may compile and even be true, but its proof obligation does not constrain the original side-effecting behavior.

Axioms that misrepresent the API contract. `axiom Redis` : Type captures *some* Redis but not the real one. The agent does not encode Redis’s consistency guarantees, ordering, or failure modes. A theorem proven over the axiomatized interface may not transfer to the live system if the production code relies on those omitted guarantees.

C FVSpec:FV Grader Distributions

8 shows difficulty distribution according to the Haiku 4.5 grader. 9 shows difficulty grade against output characteristics and LoC.

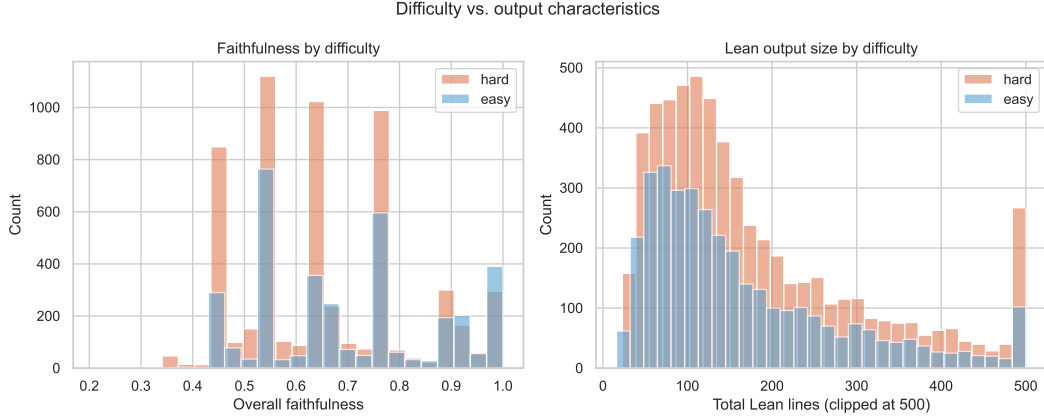


Figure 9: Difficulty vs. output characteristics: overall faithfulness (left) and total Lean lines (right), stratified by difficulty.

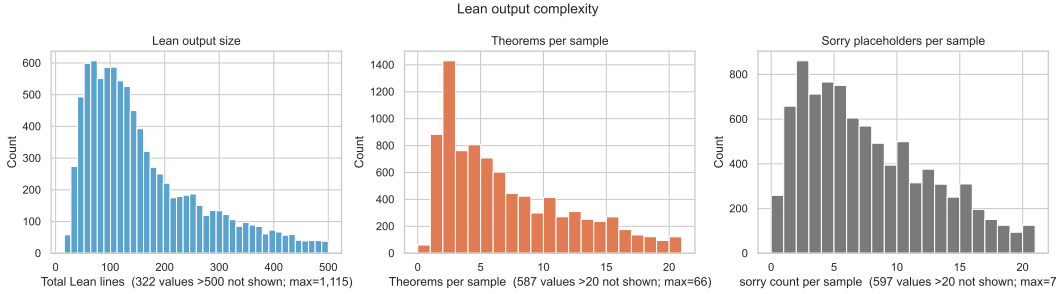


Figure 10: Lean output complexity: total lines (left), theorems per sample (center), and sorry placeholders per sample (right). Tail counts and maxima shown in axis labels.

D FVSpec:FV Number of Theorems and Complexity per Sample

10 shows the median sample contains multiple theorems each requiring an independent proof, and the sorry count tracks the theorem count closely, as expected from our pipeline design.

E FVSpec:PBT Code Quality Metrics

Figure 11 summarizes three code-quality metrics across the corpus. The source tests span a wide range of complexity, from trivial single-assertion tests to multi-strategy tests exercising complex library APIs—diversity inherited directly from real-world engineering practice and not curated for difficulty.

F FVSpec:FV Pipeline Cost

Per-sample pipeline cost is right-skewed (Figure 12): most samples compile within a few agent turns, but a tail of difficult translations requires many repair iterations before the Lean LSP reports success or the budget is exhausted.

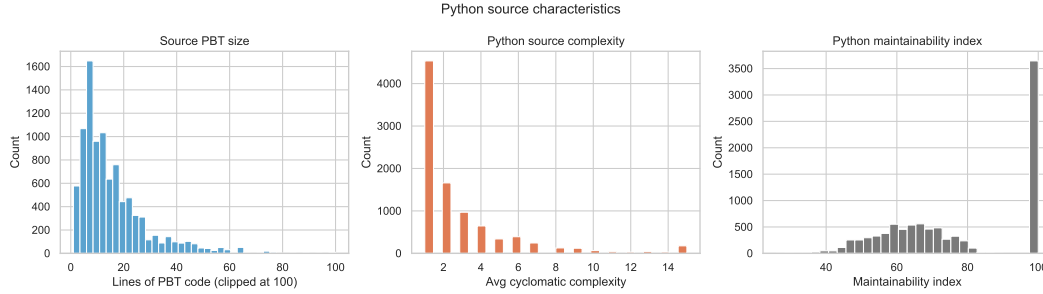


Figure 11: Code-quality metrics for the 11,039 deduplicated PBTs, computed by Radon [1]. *Left*: lines of code. *Center*: cyclomatic complexity—the number of linearly independent execution paths through the function, i.e. one plus the number of binary branch points [35]; higher values indicate more complex control flow. *Right*: maintainability index—a 0–100 composite of Halstead volume [17], cyclomatic complexity, and source lines, where higher is more maintainable; the spike at 100 is an artefact of Radon capping the score at that value.

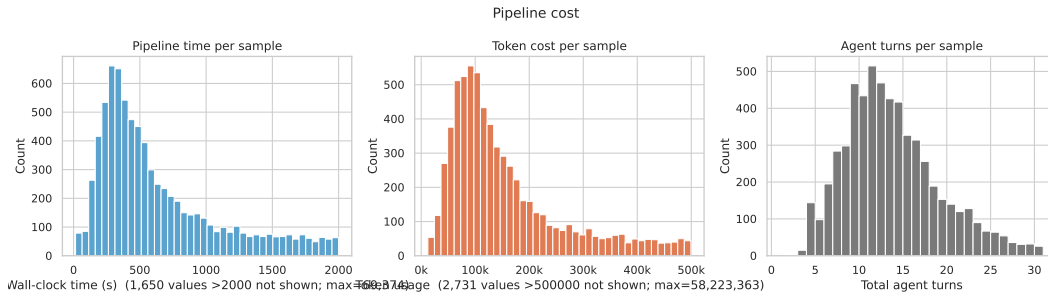


Figure 12: Pipeline cost per sample: wall-clock time (left), token usage (center), and agent turns (right). Tail counts and maxima shown in axis labels.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The main claims made in the abstract concern the properties of the new FVSpec:PBT and FVSpec:FV benchmark datasets, which we have constructed. The body of the paper describes exactly the construction process we used for these datasets, and the experiments we have performed that justify the major claims we make about these benchmarks.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: In section 5 and appendix B.2 discuss the potential of uninteresting data via axiomatic interfaces to dependencies and what we do to measure, confront, and filter this.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The main artifacts associated with our paper are the FVSpec:PBT and FVSpec:FV datasets. We describe the construction process for this dataset, which uses an open source translation harness, also an artifact released with this paper. Our construction process and baseline experiments were performed with publicly available AI models, and the paper describes the process in detail. As a result, we believe all the information and code needed to replicate the experiments are available to reviewers (albeit, full replication would require substantial token cost).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We use well documented uv packages (following the pyproject.toml standard with a lockfile) for translation generation pipeline and baselines. Given a uv and elan install, the user only needs to supply LLM provider API keys to reproduce everything.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The `config.toml` files in the code artifact declare every variable needed for reproduction down to random seed. We additionally recommend reading each CLI's `-help` menu for guidance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The phenomena at play in these results are always either quite qualitative or binary. We largely found that doing `pass@5` in baselines did not leave an interesting distribution across the 5 attempts, so it made the most sense to keep the best and throw out the rest as is the `pass@k` standard. In terms of benchmark generation, a similar principle applies: comparisons between multiple formalizations of the same PBT do not reveal interesting information when viewed from a statistical perspective, rather what we showed with our metrics gets to the point.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Figure 7 reports walltime and token usage for baselines, and 12 reports those plus agent turns per sample. There were lots of prototype runs that consumed (ballpark) roughly 3-10% of the compute that the "actual" runs consumed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: we read the ethics guidelines and it's all irrelevant to us.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We briefly discuss the upside for cyberhardening that cheap automatic proofs would provide, and how evals like this help bring that world about. We cite position papers that explain this worldview in more detail.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: Though part of our dataset is scraped, I'm having trouble figuring out how we may have made lives easier for some attacker or harm-doer.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: FVSpec:PBT inherits the license of the originating repos. We have filtered all repos marked with non-permissive licenses. FVSpec:FV is a novel dataset constructed by AI means which we plan to release MIT-Apache2 dual.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- 786
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- 787
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.
- 788
- 789

790 **13. New assets**

791 Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

792

793 Answer: [\[Yes\]](#)

794 Justification: Yes, the two datasets will be attached in openreview alongside the anonymized code artifact.

795

796 Guidelines:

- 797
- The answer [\[N/A\]](#) means that the paper does not release new assets.
 - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
 - The paper should discuss whether and how consent was obtained from people whose asset is used.
 - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.
- 798
- 799
- 800
- 801
- 802
- 803
- 804

805 **14. Crowdsourcing and research with human subjects**

806 Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

807

808

809 Answer: [\[N/A\]](#)

810 Justification: We did none of that.

811 Guidelines:

- 812
- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
 - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
 - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.
- 813
- 814
- 815
- 816
- 817
- 818
- 819

820 **15. Institutional review board (IRB) approvals or equivalent for research with human subjects**

821

822 Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

823

824

825

826 Answer: [\[N/A\]](#)

827 Justification: We did none of that.

828 Guidelines:

- 829
- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
 - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
 - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- 830
- 831
- 832
- 833
- 834
- 835
- 836

837 • For initial submissions, do not include any information that would break anonymity (if
838 applicable), such as the institution conducting the review.

839 **16. Declaration of LLM usage**

840 Question: Does the paper describe the usage of LLMs if it is an important, original, or
841 non-standard component of the core methods in this research? Note that if the LLM is used
842 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
843 scientific rigor, or originality of the research, declaration is not required.

844 Answer: [\[Yes\]](#)

845 Justification: The paper is largely about agent behavior and extends a now standard "loop"
846 scaffold architecture with LSP tool calls, and the paper describes LLM usage very thor-
847 oughly.

848 Guidelines:

849 • The answer [\[N/A\]](#) means that the core method development in this research does not
850 involve LLMs as any important, original, or non-standard components.

851 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
852 be described.